

Efficient Filtered Vector Search with Diverse Labels: A Trie-Free and Adaptive Routing Approach: The Supplement

TABLE I: Feature importance scores in experiments.

Feature	p_{pass}	$ F_{pass} $	$ L_q $
Importance score	0.7088	0.1889	0.1023

I. ROUTER TRAINING

The main contents show that different LANNs queries favor different search paradigms. Based on this observation, ALPS uses an XGBoost-based router to select among three search paradigms: unified-graph search, proximity-graph search, and pre-filtering brute-force search. The router uses three query-level features, i.e., p_{pass} , $|F_{pass}|$, and $|L_q|$, which capture both vector selectivity and label-set structure. Table I shows that all three features receive non-negligible importance scores on real datasets, indicating that each feature contributes to the routing decisions. We implement the router with XGBoost because it can capture nonlinear feature interactions while maintaining low inference latency. In our implementation, the maximum tree depth is set to 6, the learning rate is set to 0.05, and the number of boosting rounds is set to 300. All router hyperparameters were fixed before evaluation, and no test queries were used for hyperparameter tuning or early stopping.

We randomly generate 40,000 LANNs queries from all eight datasets, with 5,000 queries from each dataset. These queries exclude the test queries used in the main evaluation. For each query, we evaluate the three candidate paradigms using the same implementations and parameter settings as in the main experiments. Specifically, the unified-graph-based paradigm uses TFNG with $\delta = 6$, the same construction parameter as UNG; the proximity-graph-based paradigm adopts the FAVOR-style search strategy used by ALPS; and the pre-filtering brute-force paradigm exhaustively computes distances over all qualified vectors, thus always achieving a recall of 1. For each paradigm, we follow the same search-queue-length tuning protocol as in the QPS-recall evaluation, varying the maximum search queue length from 10 to 70,000. We then assign the query to the paradigm with the highest QPS among those whose Recall@10 is at least 0.9. If multiple paradigms have the same QPS after measurement rounding, we break ties by first choosing the one with higher recall, and then by a fixed deterministic order: unified-graph-based search, proximity-graph-based search, and pre-filtering.

We generate 5,000 training queries for each of the eight datasets and train a single routing model on the combined 40,000 queries. To evaluate the routing accuracy reported in the main text, we use a separate set of 10,000 test queries for each dataset. These test queries are disjoint from the training queries. We also

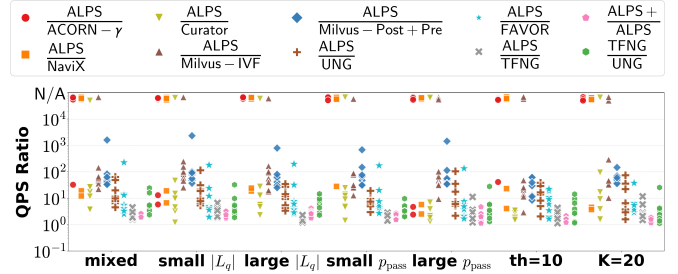


Fig. 1: QPS ratios of our approach to baseline methods for achieving recall ≥ 0.95 .

conduct a leave-one-dataset-out evaluation, where all queries from one dataset are removed from training and the trained model is tested on this held-out dataset. As shown in Tables IV and V in the main contents, the routing accuracy remains above 90% under both settings, indicating that the learned routing rule generalizes across datasets. This accuracy is acceptable, given that the resulting ALPS outperforms state-of-the-art methods in main experiments.

As shown in Table IV in the main contents, the training time cost is only a few seconds. In all experiments, query latency is measured from receiving the query to returning top- K results, including feature extraction and model prediction. The feature-extraction and prediction overhead is only a few microseconds, showing that the router is suitable for real-time query processing.

II. QPS RATIOS FOR ACHIEVING RECALL ≥ 0.95

Figure 1 reports the QPS ratios of our methods to the state-of-the-art baselines at recall ≥ 0.95 . It covers experimental settings with varying $|L_q|$, p_{pass} , th , and K , where each dot represents the result on one dataset. For a fair comparison, we compute each ratio by dividing the peak QPS of a proposed method by that of the corresponding baseline, subject to both methods achieving recall ≥ 0.95 . If a baseline cannot reach this recall under a given setting, we report the corresponding ratio as N/A. As shown in the figure 1, the QPS ratios of ALPS to the baselines are often at least 10. These results confirm our observation that, **for online queries over workloads with diverse labels, ALPS often delivers more than 10 $\times$ the QPS of state-of-the-art methods at recall ≥ 0.95 .**

III. CACHE HIT RATES COMPARISON

Frequent switching among search paradigms in ALPS may reduce cache locality and make repeated data accesses a major performance bottleneck. Therefore, ALPS+ groups queries with

TABLE II: Cache hit rates comparison (AP signifies ALPS).

Dataset	Alg.	L1 (%)	L2 (%)	L3 (%)	Dataset	Alg.	L1 (%)	L2 (%)	L3 (%)
Genome	AP	89.95	61.49	81.31	Music	AP	88.51	48.82	35.55
	AP+	89.99	83.80	91.66		AP+	90.53	58.61	73.98
Reviews	AP	84.16	44.68	39.38	BReviews	AP	82.17	60.07	31.41
	AP+	87.76	54.35	77.44		AP+	88.67	60.71	74.63
Amazon	AP	88.79	56.14	27.64	Tiktok	AP	72.88	96.95	18.16
	AP+	89.84	61.57	64.11		AP+	73.44	98.04	57.04
VImg	AP	84.32	55.75	18.42	Laion	AP	80.88	59.43	49.95
	AP+	88.15	56.39	62.47		AP+	86.46	65.00	71.59

similar execution behavior to improve memory-access locality. Table II reports the cache hit rates of ALPS and ALPS+, where L1, L2, and L3 denote the three CPU cache levels. ALPS+ achieves higher hit rates at all three levels, indicating that query grouping effectively improves cache locality and contributes to its higher throughput.